Actividad Individual: Aplicación de Lenguajes Regulares y Libres de Contexto Análisis Léxico Sintáctico y Semántico con ANTLR

Teoría de la Computación
Jorge Valenzuela

Resumen

Este documento sirve como guía para la actividad individual sobre la aplicación de los autómatas finitos (lenguajes regulares) y autómatas de pila (lenguajes libres de contexto) a través del uso de la herramienta **ANTLR v4**. El objetivo es definir las reglas de producción de una gramática libre de contexto y las expresiones regulares para construir un evaluador de expresiones aritméticas. A partir de las definiciones de la gramática, se generarán los respectivos analizadores léxico (lexer) y sintáctico (parser) y se extenderá mediante el patrón **Visitor** para realizar análisis semántico (cálculo del resultado).

Índice

1. Objetivos de la actividad **2**

2. Prerrequisitos **2**

3. Paso 1: Crear la Gramática **2**

4. Paso 2: Generación de Código **3**

5. Paso 3: Implementación del Visitor (Análisis Semántico) **4**

6. Paso 4: Clase Principal (Main) **5**

7. Paso 5: Compilación y Ejecución **6**

8. Ejercicios adicionales **6**

9. Referencias **6**

1 Objetivos de la actividad

- Comprender la estructura de un archivo de gramática `.g4`.
- Utilizar la herramienta ANTLR para generar código Java (Lexer y Parser).
- Implementar el patrón *Visitor* para recorrer el árbol de sintaxis abstracta (AST).
- Realizar análisis semántico simple: evaluación de expresiones aritméticas.

2 Prerrequisitos

Para realizar esta actividad, asegúrese de:

1. Repasar los conceptos de análisis léxico y sintáctico como etapas en compilación e interpretación de lenguajes.
2. Tener instalado **JDK (Java Development Kit)**: Versión 8 o superior.
3. Haber descargado la herramienta **ANTLR Tool**: El archivo `antlr-4.x-complete.jar` se descarga del sitio `https://www.antlr.org/download.html`.
4. Instalar y ajustar un ambiente de trabajo según el sistema operativo que use, siga las instrucciones correspondientes en el tutorial para comenzar
5. Contar con un editor de texto.

3 Paso 1: Crear la Gramática

Visitar el sitio de ANTLR 4: `https://www.antlr.org` y copiar la gramática que se muestra en la parte inferior izquierda del recuadro de "Quick Start", el que se muestra en la imagen

```
grammar Expr;
prog:   (expr NEWLINE)* ;
expr:   expr ('*'|'/') expr
    |   expr ('+'|'-') expr
    |   INT
    |   '(' expr ')'
    ;
NEWLINE : [\r\n]+ ;
INT     : [0-9]+ ;
```

ANTLR utiliza una notación BNF extendida, copie el texto tal cual y cree un archivo llamado `Expr.g4`, luego haga algunas modificaciones para que quede con el siguiente contenido.

```
grammar Expr;
prog:   (expr NEWLINE)* EOF ;
expr:   expr ('*'|'/') expr     #MultDiv
    |   expr ('+'|'-') expr     #AddSub
    |   INT                     #Int
    |   '(' expr ')'            #Parens
    ;
NEWLINE : [\r\n]+ ;
INT     : [0-9]+ ;
WS      : [ \t]+ -> skip ; // Ignorar espacios en blanco
```

Listing 1: Archivo Expr.g4

*Nota: se deben añadir **etiquetas** (ej: #MultDiv) a las alternativas de la regla `expr`. Esto es una extensión recomendada para facilitar la implementación del Visitor, generando métodos específicos para cada tipo de operación.*

Explicación de la Gramática

- `prog`: Primera regla y punto de entrada. Espera una o más expresiones seguidas de salto de línea.
- `expr`: Regla recursiva. ANTLR 4 maneja la recursividad por la izquierda automáticamente. El orden de las alternativas define la precedencia (Multiplicación y División antes que Suma y Resta).
- `NEWLINE`, `INT`: Reglas léxicas para (tokens), escritas como expresiones regulares.
- `WS`: Regla para espacios que se ignoran y descartan (`->skip`).

4 Paso 2: Generación de Código

Abra una terminal en la carpeta donde guardó `Expr.g4` y ejecute el siguiente comando (asegúrese de tener el `jar` de ANTLR en la misma carpeta o en su `CLASSPATH`):

```
java -jar antlr-4.13.1-complete.jar Expr.g4 -visitor -no-listener
```

Si siguió el tutorial de instalación y creo alias para los comandos "antlr4" y "grun" puede utilizarlos para abreviar el comando, así:

```
antlr4  Expr.g4 -visitor -no-listener
```

Opciones usadas:

- `-visitor`: Genera las clases base para el patrón Visitor (`ExprBaseVisitor.java` y `ExprVisitor.java`).
- `-no-listener`: No generamos código para el patrón Listener (ahorramos la generación de archivos que no usaremos en la actividad).

Nota: El patron `visitor` es un patrón de diseño que permite definir nuevas operaciones sobre una jerarquía de objetos sin modificar las clases de esa jerarquía. Se usará para recorrer el árbol sintáctico y añadir la lógica de la evaluación.

La generación producirá los siguientes archivos Java:

- `ExprLexer.java`: Analizador léxico.
- `ExprParser.java`: Analizador sintáctico.
- `ExprBaseVisitor.java`: Clase base para implementar nuestra lógica.
- `ExprVisitor.java`: Interfaz del visitor.

5 Paso 3: Implementación del Visitor (Análisis Semántico)

Para evaluar la expresión, crearemos una clase que herede de `ExprBaseVisitor`. Esta clase actuará como una **gramática de atributos**, donde cada nodo del árbol devuelve un valor (el resultado parcial de la operación).

Cree el archivo `EvalVisitor.java`:

```
import java.util.*;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.*;
// Heredamos de ExprBaseVisitor<Integer> porque las expresiones retornan enteros
public class EvalVisitor extends ExprBaseVisitor < Integer > {
  @Override
  public Integer visitInt(ExprParser.IntContext ctx) {
    // Convertir el token INT a numero entero
    return Integer.parseInt(ctx.INT().getText());
  }

  @Override
  public Integer visitAddSub(ExprParser.AddSubContext ctx) {
    // Evaluar recursivamente la izquierda y la derecha
    int left = visit(ctx.expr(0));
    int right = visit(ctx.expr(1));

    // Determinar el operador
    //if (ctx.op.getText().equals("+")) {
    if (ctx.getChild(1).getText().equals("+")) {
      return left + right;
    } else {
      return left - right;
    }
  }

  @Override
  public Integer visitMultDiv(ExprParser.MultDivContext ctx) {
    int left = visit(ctx.expr(0));
    int right = visit(ctx.expr(1));

    //if (ctx.op.getText().equals("*")) {
    if (ctx.getChild(1).getText().equals("*")) {
      return left * right;
    } else {
      return left / right;
    }
  }

  @Override
  public Integer visitParens(ExprParser.ParensContext ctx) {
    // Simplemente retornar el valor de la expresion interior
    return visit(ctx.expr());
  }
}
```

Listing 2: Archivo EvalVisitor.java

Nota sobre el operador: Estamos usando la gramática exacta del enunciado sin etiquetas de tokens, con la instrucción `ctx.getChild(1).getText()` para obtener el operador correspondiente en `visitAddSub` y `visitMultDiv`.

6 Paso 4: Clase Principal (Main)

Necesitamos un punto de entrada que lea la consola, invoque al parser y luego al visitor. Cree el archivo `Main.java`:

```
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.*;
import java.io.*;
public class Main {
  public static void main(String[] args) throws Exception {
    // 1. Crear un flujo de entrada desde la consola
    CharStream input = CharStreams.fromStream(System.in);
    // 2. Crear el Lexer
    ExprLexer lexer = new ExprLexer(input);

    // 3. Crear un buffer de tokens
    CommonTokenStream tokens = new CommonTokenStream(lexer);

    // 4. Crear el Parser
    ExprParser parser = new ExprParser(tokens);

    // 5. Eliminar errores de recuperacion comunes para ver fallos claros
    parser.removeErrorListeners();
    parser.addErrorListener(new BaseErrorListener() {
      public void syntaxError(Recognizer < ? , ? > recognizer,
        Object offendingSymbol, int line,
        int charPositionInLine, String msg,
        RecognitionException e) {
        System.out.println("Error de sintaxis en linea " + line + ":" +
    charPositionInLine + " " + msg);
      }
    });

    // 6. Iniciar el parseo desde la regla 'prog'
    ParseTree tree = parser.prog();

    // 7. Si no hay errores de sintaxis, visitar el arbol
    if (parser.getNumberOfSyntaxErrors() == 0) {
      EvalVisitor eval = new EvalVisitor();
      // Visitamos el arbol. Como prog puede tener varias expresiones,
      // iteramos sobre los hijos del nodo raiz.
      for (int i = 0; i < tree.getChildCount(); i++) {
        ParseTree exprTree = tree.getChild(i);
        if (exprTree instanceof ExprParser.ExprContext) {
          Integer result = eval.visit((ExprParser.ExprContext) exprTree);
          System.out.println("Resultado: " + result);
        }
      }
    }
  }
}
```

Listing 3: Archivo Main.java

7 Paso 5: Compilación y Ejecución

Para compilar y ejecutar, necesita incluir el `jar` de ANTLR en el *classpath*.

En Linux / macOS

```
Compilar
javac -cp .:antlr-4.13.1-complete.jar *.java
Ejecutar
java -cp .:antlr-4.13.1-complete.jar Main
```

En Windows (CMD)

```
Compilar
javac -cp .;antlr-4.13.1-complete.jar *.java
Ejecutar
java -cp .;antlr-4.13.1-complete.jar Main
```

Prueba

Una vez ejecutado `Main`, escriba expresiones y presione control-Z en windows o control-D en unix / linux:

```
10 + 2 * 3
Resultado: 16
(10 + 2) * 3
Resultado: 36
5 / 2
Resultado: 2
```

8 Ejercicios adicionales

Para profundizar en la actividad, intente realizar las siguientes modificaciones:

1. **Soporte para Decimales**: Modifique la gramática y el Visitor para usar `double` en lugar de `int`.
2. **Manejo de Errores Semánticos**: Agregue una validación en el Visitor para detectar división por cero y lanzar una excepción personalizada.
3. **Variables**: Extienda la gramática para permitir asignaciones (ej: `x = 10`) y uso de variables en expresiones. Necesitará un `Map<String, Integer>` en el Visitor para guardar el estado.

9 Referencias

- ANTLR v4 Documentation: `https://github.com/antlr/antlr4`
- The Definitive ANTLR 4 Reference, Terence Parr.
- Notas de clase para lenguajes regulares y libres de contexto
- Notas de clase sobre autómatas finitos, expresiones regulares y autómatas de pila